
Filière : 4^{ème} année — Intelligence Artificielle et Data

Module : Deep Learning

SYNTHÈSE DU PROJET DE FIN DE MODULE

Conception, implémentation, comparaison et analyse
critique de modèles de deep learning

Réalisée par : Imane Nadif

Encadrant : Mosaab Batal

Groupe : G2 — Moulay Youssef

Année universitaire 2025–2026

Table des matières

1	Vue d'ensemble	3
2	Partie I — MLP et ingénierie PyTorch	3
2.1	Concepts fondamentaux	3
2.1.1	nn.Module, paramètres, gradient, state_dict, device	3
2.2	Dataset : Wine Quality	4
2.3	Implémentation : nn.Sequential vs classe personnalisée	4
2.4	Trois stratégies d'initialisation	4
2.5	Test des 5 architectures et sélection du Best Model	5
2.6	Évaluation du Best Model	5
2.7	Question de synthèse — Partie I	6
3	Partie II — CNN et architectures dérivées	7
3.1	Limites du MLP pour les images	7
3.2	Les 3 idées fondatrices des CNN	7
3.3	Corrélation croisée 2D et calculs manuels	7
3.4	Canaux multiples	7
3.5	Convolution 1×1	7
3.6	Architectures CNN implémentées	8
3.6.1	LeNet	8
3.6.2	AlexNet simplifié	8
3.6.3	VGG simplifié	8
3.7	Visualisation et interprétation des feature maps	8
3.8	Comparaison MLP vs CNN vs Dérivés	9
3.9	Question de synthèse — Partie II	10
4	Partie III — RNN / LSTM / GRU / Transformers	10
4.1	Modèles de langage	10
4.1.1	Objectif probabiliste et règle de chaîne	10
4.1.2	Approches n-grammes	10
4.1.3	Perplexité	11
4.2	Réseaux de neurones récurrents (RNN)	11
4.2.1	Idée centrale	11
4.2.2	BPTT et gradient clipping	11
4.3	LSTM : Long Short-Term Memory	12
4.3.1	Pourquoi les LSTM ?	12
4.3.2	Équations	12
4.3.3	Rôle des portes	12
4.4	GRU : Gated Recurrent Unit	13
4.4.1	Principe	13
4.4.2	Comparaison LSTM / GRU	13
4.5	Variantes architecturales : RNN profonds et bidirectionnels	13
4.5.1	RNN profonds	13
4.5.2	Dropout récurrent	13
4.5.3	RNN bidirectionnels	13
4.6	Préparation des données — Tatoeba fra-eng	14
4.7	Architecture encodeur-décodeur	14

4.7.1	Idée générale	14
4.7.2	Seq2Seq récurrent (encodeur GRU + décodeur GRU)	14
4.8	Décodage : glouton et beam search	15
4.8.1	Décodage glouton	15
4.8.2	Score d'une séquence	15
4.8.3	Beam search	15
4.8.4	Pénalisation de longueur	15
4.9	Transformer — <i>Attention is all you need</i>	16
4.9.1	Mécanisme d'attention	16
4.9.2	Positional encoding	16
4.9.3	Masque causal	16
4.10	Évaluation : Perplexité et BLEU	16
4.11	Comparaison globale des modèles séquentiels	18
4.12	Mini-résumé opérationnel	18
4.13	Question de synthèse — Partie III	18
5	Question transversale finale	19
5.1	Tableau de synthèse général	19
5.2	Conclusion	20

1. Vue d'ensemble

Ce projet couvre trois grandes familles d'architectures de deep learning, chacune adaptée à un type de données spécifique. La progression suit une logique pédagogique claire : données tabulaires (MLP), données structurées spatialement (CNN + dérivés), puis données séquentielles (RNN, LSTM, GRU, Transformers).

Une chaîne de traitement typique est la suivante :

1. Représenter les données sous forme de tenseurs
2. Définir un modèle adapté à la structure des données
3. Entraîner le modèle par optimisation d'une perte
4. Évaluer et interpréter les résultats
5. Comparer les architectures de manière critique

Principe unificateur

Un même principe (descente de gradient, rétropropagation) s'applique à toutes les architectures. Ce qui change est l'**inductive bias** : la façon dont chaque architecture exploite la structure géométrique, locale ou temporelle des données.

Partie	Architecture	Type de données	Dataset
I	MLP	Tabulaires	Wine Quality
II	CNN + Dérivés	Images	MNIST
III	RNN / LSTM / GRU / Transformer	Séquences	Tatoeba fra-eng

2. Partie I — MLP et ingénierie PyTorch

2.1. Concepts fondamentaux

2.1.1. nn.Module, paramètres, gradient, state_dict, device

En PyTorch, tout réseau hérite de `nn.Module`. Un **paramètre** est un tenseur avec `requires_grad=True`. Lors de `loss.backward()`, les gradients sont calculés via la règle de la chaîne :

$$\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial W}, \quad W \leftarrow W - \eta \cdot \frac{\partial \mathcal{L}}{\partial W}$$

Propagation avant :

$$x \xrightarrow{W_1, b_1} z_1 \xrightarrow{\text{ReLU}} a_1 \xrightarrow{W_2, b_2} z_2 \xrightarrow{\text{ReLU}} a_2 \xrightarrow{W_3, b_3} \hat{y}$$

Le `state_dict()` est un dictionnaire ordonné de tous les poids :

```
1 torch.save(model.state_dict(), 'meilleur_mlp.pth')
2 model.load_state_dict(torch.load('meilleur_mlp.pth',
3                               map_location=device))
3 model.eval()
```

Listing 1 – Sauvegarde et rechargement du meilleur modèle

Le modèle **et** les données doivent être sur le même device :

```

1 device = torch.device('cuda' if torch.cuda.is_available()
   else 'cpu')
2 model  = model.to(device)
3 X_batch = X_batch.to(device)
4 y_batch = y_batch.to(device)

```

Listing 2 – Vérification de cohérence device

Le **Dropout** désactive aléatoirement une fraction p des neurones pendant l'entraînement. Actif en `model.train()`, désactivé en `model.eval()`.

2.2. Dataset : Wine Quality

178 exemples, 13 features chimiques, 3 classes. Pipeline : normalisation `StandardScaler` (fit uniquement sur train), split stratifié 70 %/15 %/15 %, `DataLoaders` de taille 32.

2.3. Implémentation : `nn.Sequential` vs classe personnalisée

```

1 class MLP(nn.Module):
2     def __init__(self, input_size, hidden_sizes, output_size,
3                 activation='relu', dropout_rate=0.0):
4         super().__init__()
5         self.layers = nn.ModuleList()
6         self.dropouts = nn.ModuleList()
7         prev = input_size
8         for h in hidden_sizes:
9             self.layers.append(nn.Linear(prev, h))
10            self.dropouts.append(nn.Dropout(p=dropout_rate))
11            prev = h
12        self.output_layer = nn.Linear(prev, output_size)
13        self.activation = nn.ReLU()
14
15        def forward(self, x):
16            for layer, drop in zip(self.layers, self.dropouts):
17                x = drop(self.activation(layer(x)))
18            return self.output_layer(x)

```

Listing 3 – Classe MLP personnalisée avec Dropout optionnel

2.4. Trois stratégies d'initialisation

Stratégie	Formule	Analyse
Gaussienne	$W \sim \mathcal{N}(0, 0.01)$	Convergence lente
Constante	$W = 1.0$	Symétrie impossible à briser
Xavier	$W \sim U\left(-\frac{\sqrt{6}}{\sqrt{n_{in} + n_{out}}}, +\frac{\sqrt{6}}{\sqrt{n_{in} + n_{out}}}\right)$	Stabilise la variance — recommandé

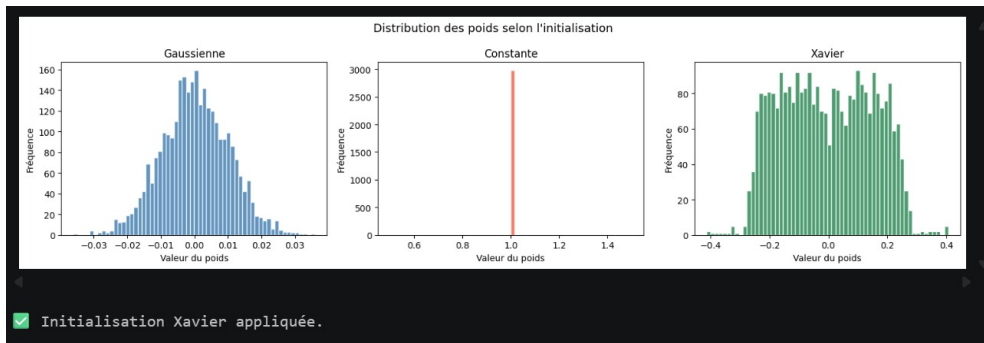


FIGURE 1 – Distribution des poids : Gaussienne, Constante, Xavier

2.5. Test des 5 architectures et sélection du Best Model

Architecture	Couches cachées	Paramètres	Val Acc.	Test Acc.
MLP 1	[32]	547	100.00%	85.19%
MLP 2	[64, 32]	3 075	100.00%	100.00%
MLP 3	[128, 64, 32]	12 227	100.00%	96.30%
MLP 4	[256, 128, 64]	44 931	100.00%	96.30%
MLP 5	[512, 256, 128, 64]	179 843	100.00%	88.89%
Best Model	[64, 32]	3 075	100.00%	100.00%

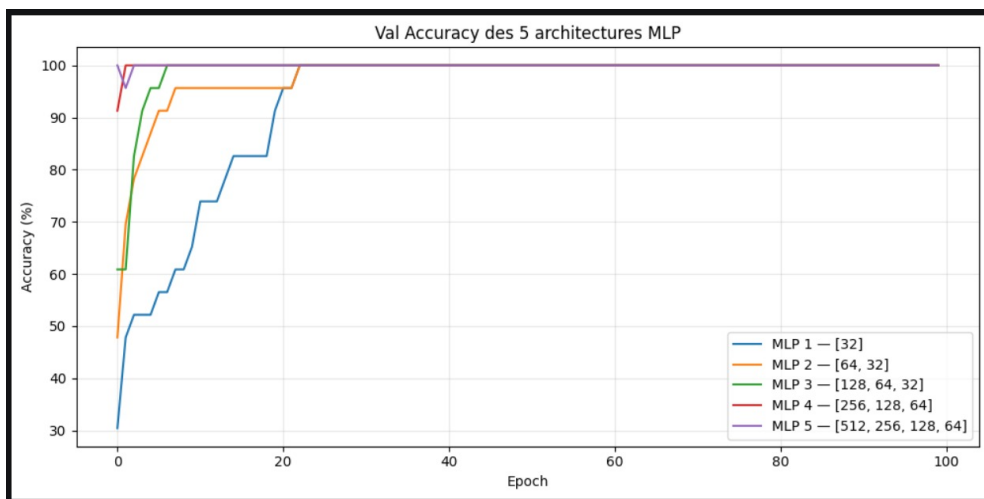


FIGURE 2 – Comparaison Val Accuracy des 5 architectures MLP (Wine Quality)

2.6. Évaluation du Best Model

Métrique	Valeur	Interprétation
Accuracy	100.00%	Proportion de prédictions correctes
Precision	100.00%	Parmi les prédictions positives, combien sont correctes ?
Recall	100.00%	Parmi les vrais positifs, combien sont détectés ?
F1-score	100.00%	Moyenne harmonique precision/recall

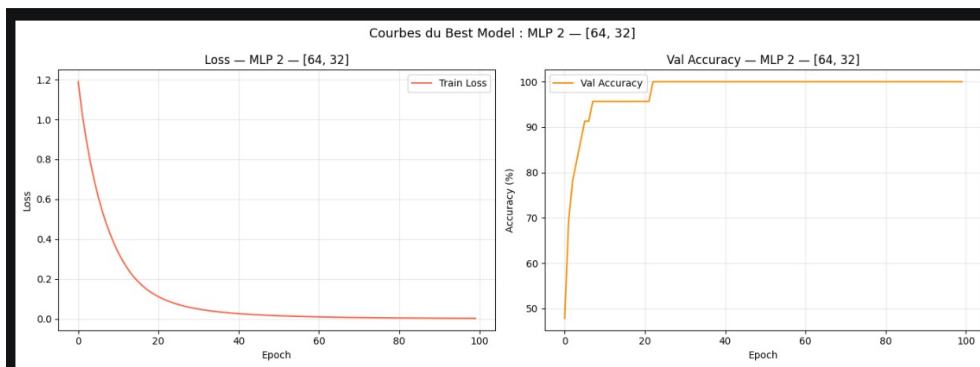


FIGURE 3 – Courbes Loss et Val Accuracy du Best Model : MLP 2 — [64, 32]

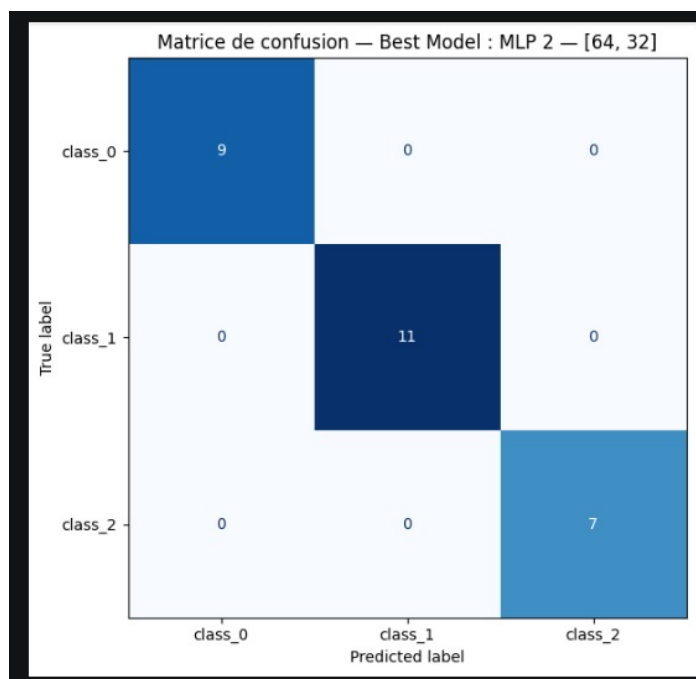


FIGURE 4 – Matrice de confusion du Best Model (MLP 2 — [64, 32]) sur le test set

2.7. Question de synthèse — Partie I

Question

Dans quelle mesure un MLP bien paramétré constitue-t-il une solution pertinente pour la classification tabulaire, et quelles sont ses principales limites ?

Le MLP est bien adapté aux données tabulaires car les features n'ont pas de structure spatiale ni temporelle. Parmi les 5 architectures testées, le Best Model est l'architecture MLP 2 — [64, 32] avec 3075 paramètres et une test accuracy de 100%. L'initialisation Xavier donne les meilleurs résultats car elle stabilise la variance. L'initialisation constante échoue à cause du problème de symétrie.

Limites : aucun inductive bias structurel, risque de surapprentissage sur petit dataset (178 exemples), inadapté aux images et aux séquences.

3. Partie II — CNN et architectures dérivées

3.1. Limites du MLP pour les images

Image MNIST $28 \times 28 = 784$ pixels. Si on aplatit : perte de la structure spatiale, explosion des paramètres ($784 \times 256 = 200\,704$), pas d'invariance aux translations.

3.2. Les 3 idées fondatrices des CNN

1. **Localité** : le filtre ne regarde qu'une petite région
2. **Partage des poids** : le même filtre glisse sur toute l'image
3. **Hiérarchie** : bords $\rightarrow$ formes $\rightarrow$ objets

3.3. Corrélation croisée 2D et calculs manuels

$$S(i, j) = \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} X(i+m, j+n) \cdot K(m, n), \quad H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} - K + 2P}{S} \right\rfloor + 1$$

Exemple numérique :

$$X = \begin{pmatrix} 1 & 2 & 0 & 1 \\ 3 & 1 & 2 & 2 \\ 0 & 1 & 3 & 1 \\ 2 & 2 & 1 & 0 \end{pmatrix}, \quad K = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \Rightarrow \text{Sortie} = \begin{pmatrix} 2 & 4 & 2 \\ 4 & 4 & 3 \\ 2 & 2 & 3 \end{pmatrix}$$

Tableau des tailles de sortie (entrée 28×28 , kernel 5×5) :

padding	stride	H_{out}	Calcul
0	1	24	$\lfloor (28 - 5)/1 \rfloor + 1$
2	1	28	$\lfloor (28 - 5 + 4)/1 \rfloor + 1$
0	2	12	$\lfloor (28 - 5)/2 \rfloor + 1$
2	2	14	$\lfloor (28 - 5 + 4)/2 \rfloor + 1$

Taille après max-pooling 2×2 , stride= 2 : $\lfloor (28 - 2)/2 \rfloor + 1 = 14$.

3.4. Canaux multiples

Avec C_{in} canaux et C_{out} filtres : $S(i, j) = \sum_c \sum_m \sum_n X_c(i+m, j+n) \cdot K_c(m, n)$.
Paramètres (sans biais) : $C_{\text{out}} \times C_{\text{in}} \times k_h \times k_w$.

3.5. Convolution 1×1

Opère sur tous les canaux sans toucher la taille spatiale. Réduit le nombre de canaux, ajoute de la non-linéarité. Utilisé dans Inception et ResNet.

3.6. Architectures CNN implémentées

3.6.1. LeNet

Couche	Entrée	Opération	Sortie
Conv1	[B,1,28,28]	Conv2d(1→6, k=5, p=2)+ReLU	[B,6,28,28]
Pool1	[B,6,28,28]	MaxPool2d(2,2)	[B,6,14,14]
Conv2	[B,6,14,14]	Conv2d(6→16, k=5)+ReLU	[B,16,10,10]
Pool2	[B,16,10,10]	MaxPool2d(2,2)	[B,16,5,5]
FC1	[B,400]	Linear(400→120)+ReLU	[B,120]
FC2	[B,120]	Linear(120→84)+ReLU	[B,84]
FC3	[B,84]	Linear(84→10)	[B,10]

3.6.2. AlexNet simplifié

Ajout de **BatchNorm** (stabilise l'entraînement), **Dropout** $p = 0.5$ (réduit le surapprentissage), plus de filtres (32, 64, 128).

3.6.3. VGG simplifié

Blocs de 2 convolutions 3×3 empilées avant chaque pooling. Deux conv 3×3 couvrent le même champ récepteur qu'une conv 5×5 avec moins de paramètres et plus de non-linéarité.

```

1 nn.Conv2d(32, 32, kernel_size=3, padding=1), nn.ReLU(),
2 nn.Conv2d(32, 32, kernel_size=3, padding=1), nn.ReLU(),
3 nn.MaxPool2d(2, 2),

```

Listing 4 – Bloc VGG : deux conv 3x3 empilées

3.7. Visualisation et interprétation des feature maps

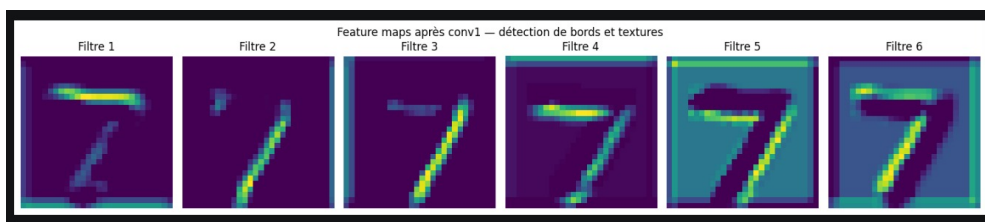


FIGURE 5 – Feature maps après la première couche de convolution (6 filtres, 28×28)

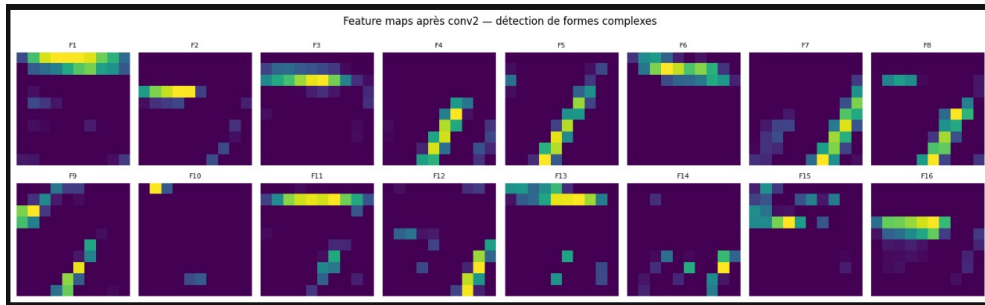


FIGURE 6 – Feature maps après la deuxième couche de convolution (16 filtres, 10×10)

Hiérarchie des représentations

conv1 = bas niveau (bords et contours) → conv2 = niveau intermédiaire (coins, courbes) → couches FC = niveau sémantique (classe).

3.8. Comparaison MLP vs CNN vs Dérivés

Modèle	Paramètres	Test Accuracy
MLP	235 146	97.45%
LeNet	61 706	98.89%
AlexNet simplifié	1 701 322	99.30%
VGG simplifié	870 634	99.28%

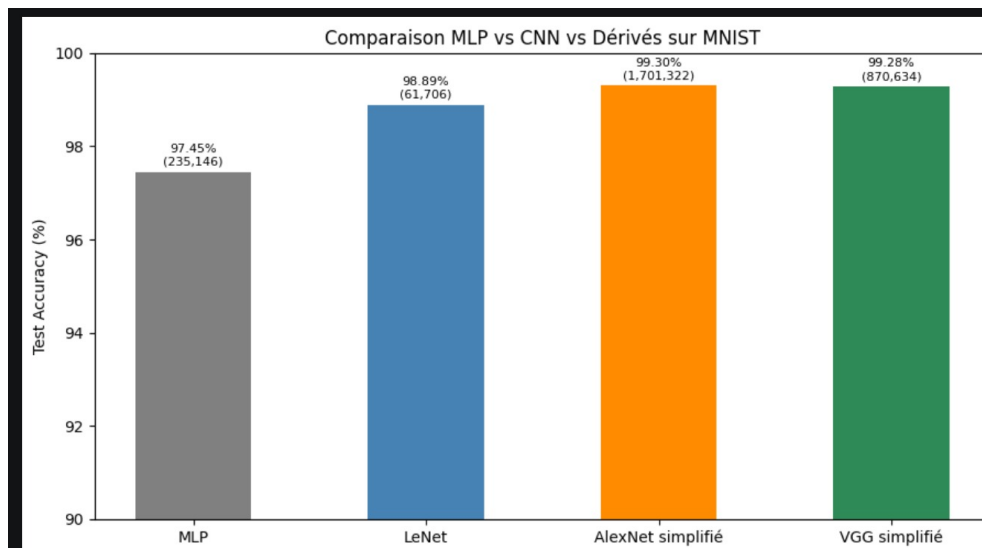


FIGURE 7 – Comparaison Test Accuracy : MLP vs CNN vs Dérivés (MNIST)

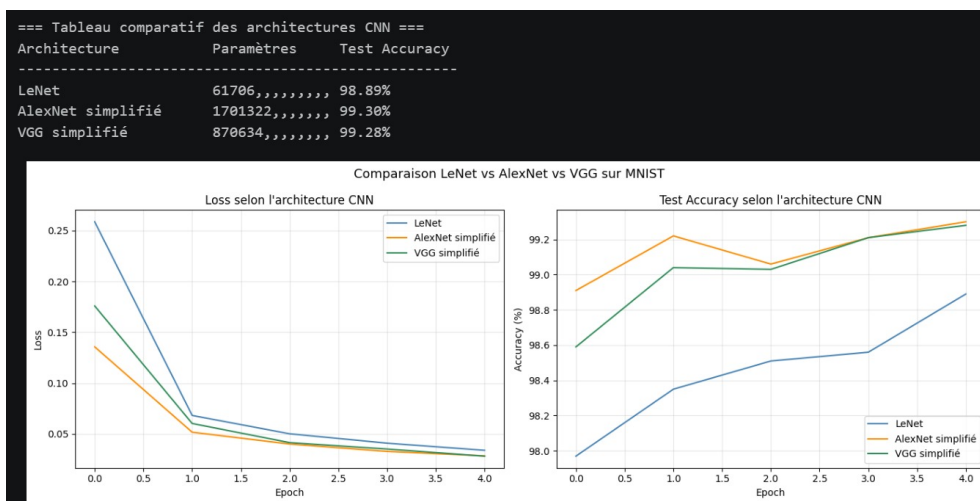


FIGURE 8 – Courbes d'apprentissage : LeNet vs AlexNet vs VGG

3.9. Question de synthèse — Partie II

Question

Pourquoi un CNN est-il plus pertinent qu'un MLP pour les images, et comment les architectures dérivées améliorent-elles les performances ?

Le CNN exploite la structure spatiale. Sur MNIST il obtient 98.89% avec 61 706 paramètres contre 97.45% pour le MLP. AlexNet ajoute BatchNorm et Dropout : 99.30%. VGG empile des conv 3×3 : 99.28%. Le max-pooling conserve les activations dominantes. La conv 1×1 réduit les canaux sans coût spatial.

4. Partie III — RNN / LSTM / GRU / Transformers

4.1. Modèles de langage

4.1.1. Objectif probabiliste et règle de chaîne

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_1, \dots, x_{t-1})$$

Pour modéliser une séquence entière, il suffit de prédire le prochain token à partir du passé.

4.1.2. Approches n-grammes

Dans les modèles de Markov d'ordre $n-1$: $P(x_t \mid x_1, \dots, x_{t-1}) \approx P(x_t \mid x_{t-n+1}, \dots, x_{t-1})$.

Bigramme : $P(x_1, \dots, x_T) \approx P(x_1) \prod_{t=2}^T P(x_t \mid x_{t-1})$.

Limites : explosion combinatoire, sparsité, incapacité à capturer de longues dépendances $\rightarrow$ d'où l'intérêt des RNN.

4.1.3. Perplexité

$$\text{PPL} = \exp\left(-\frac{1}{T} \sum_{t=1}^T \log P(x_t \mid x_{<t})\right)$$

Plus la perplexité est faible, meilleur est le modèle. PPL = taille du vocabulaire $\rightarrow$ modèle aléatoire.

4.2. Réseaux de neurones récurrents (RNN)

4.2.1. Idée centrale

Un RNN possède un état caché qui se transmet de pas en pas :

$$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b_h), \quad \hat{y}_t = \text{softmax}(W_{hq} h_t + b_q)$$

Interprétation condensée : $\tilde{x}_t = \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix}$, $h_t = \varphi(W\tilde{x}_t + b)$.

```

1 rnn = nn.RNN(input_size=32, hidden_size=64, batch_first=True)
2 X    = torch.randn(16, 10, 32)    # batch=16, longueur=10, dim
   =32
3 H0   = torch.zeros(1, 16, 64)
4 Y, Hn = rnn(X, H0)
5 print(Y.shape)    # (16, 10, 64)
6 print(Hn.shape)   # (1, 16, 64)
```

Listing 5 – RNN minimal en PyTorch

4.2.2. BPTT et gradient clipping

La BPTT déroule le RNN sur T pas. Le gradient d'une perte finale par rapport à un état antérieur contient une chaîne de jacobiens :

$$\frac{\partial L}{\partial h_k} = \sum_{t \geq k} \frac{\partial L}{\partial h_t} \prod_{i=k+1}^t \frac{\partial h_i}{\partial h_{i-1}}$$

Gradient évanescent : le produit tend vers zéro. **Gradient explosif** : le produit devient très grand.

Gradient clipping : $g \leftarrow \min\left(1, \frac{\theta}{\|g\|}\right) g$

```

1 loss.backward()
2 torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm
   =1.0)
3 optimizer.step()
```

Listing 6 – Gradient clipping en PyTorch

BPTT tronquée : au lieu de rétropropager sur toute la séquence, on coupe l'horizon temporel après τ pas. Réduit le coût mémoire au prix d'une vision plus courte du passé.

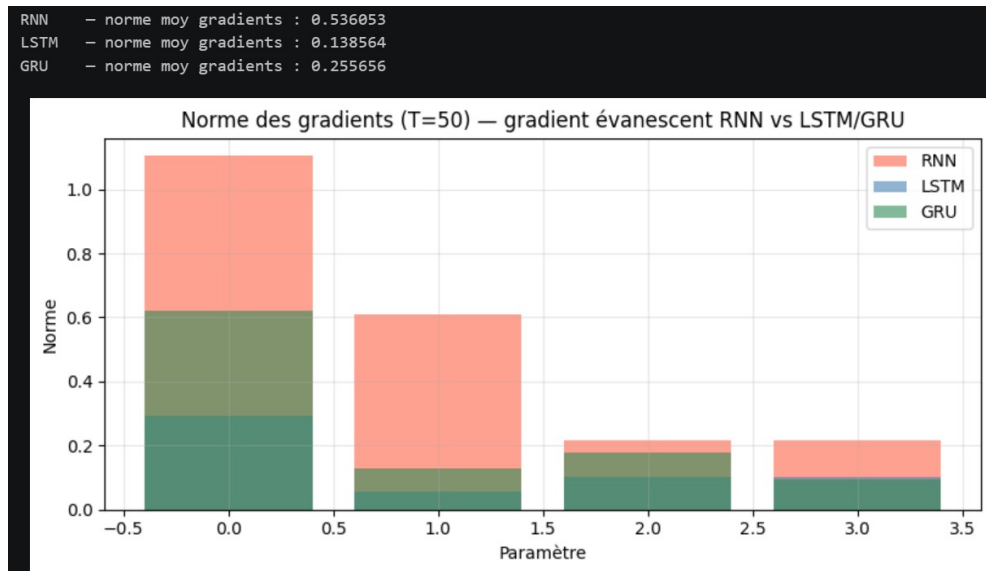


FIGURE 9 – Norme des gradients (T=50) : gradient évanescent RNN vs LSTM/GRU

4.3. LSTM : Long Short-Term Memory

4.3.1. Pourquoi les LSTM ?

Les LSTM ont été conçus pour conserver l'information utile sur de longues distances. Ils introduisent un état de cellule c_t et des portes contrôlant l'écriture, l'oubli et la lecture.

4.3.2. Équations

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad (\text{porte d'oubli}) \quad (1)$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (\text{porte d'entrée}) \quad (2)$$

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c) \quad (\text{candidat}) \quad (3)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (\text{cellule mémoire}) \quad (4)$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad (\text{porte de sortie}) \quad (5)$$

$$h_t = o_t \odot \tanh(c_t) \quad (\text{état caché}) \quad (6)$$

4.3.3. Rôle des portes

f_t : quelle mémoire garder. i_t : quoi écrire. o_t : quoi exposer. La structure réduit l'évanouissement via c_t .

```
1 lstm = nn.LSTM(input_size=32, hidden_size=64, batch_first=True)
2 Y, (Hn, Cn) = lstm(X)
3 print(Y.shape)      # (8, 15, 64)
```

```
4 print(Cn.shape) # (1, 8, 64) -- cellule memoire
```

Listing 7 – LSTM en PyTorch

4.4. GRU : Gated Recurrent Unit

4.4.1. Principe

Le GRU simplifie le LSTM. Pas d'état de cellule séparé. Deux portes :

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z) \quad (\text{mise à jour}) \quad (7)$$

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r) \quad (\text{réinitialisation}) \quad (8)$$

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h) \quad (9)$$

$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t \quad (10)$$

4.4.2. Comparaison LSTM / GRU

Critère	LSTM	GRU
Structure	3 portes + cellule c_t	2 portes
Mémoire	Explicite via c_t	Intégrée à h_t
Coût	Plus élevé	Plus léger
Usage	Longues dépendances	Bon compromis

```
1 gru = nn.GRU(input_size=32, hidden_size=64, batch_first=True)
2 Y, Hn = gru(X)
```

Listing 8 – GRU en PyTorch

4.5. Variantes architecturales : RNN profonds et bidirectionnels

4.5.1. RNN profonds

Plusieurs couches récurrentes empilées. À chaque instant t :

$$h_t^{(1)} = f^{(1)}(x_t, h_{t-1}^{(1)}), \quad h_t^{(\ell)} = f^{(\ell)}(h_t^{(\ell-1)}, h_{t-1}^{(\ell)}), \quad \ell \geq 2$$

Couche basse = motifs locaux. Couche haute = représentations abstraites.

4.5.2. Dropout récurrent

Dropout entre couches pour limiter le surapprentissage. Prudence : mal placé, il perturbe la mémoire temporelle.

4.5.3. RNN bidirectionnels

Traite la séquence dans les deux sens et concatène :

$$\vec{h}_t = f(x_t, \vec{h}_{t-1}), \quad \overleftarrow{h}_t = g(x_t, \overleftarrow{h}_{t+1}), \quad h_t = [\vec{h}_t; \overleftarrow{h}_t]$$

Utile pour l'étiquetage et la reconnaissance d'entités. **Inadapté** à la génération auto-régressive (le futur est inconnu).

4.6. Préparation des données — Tatoeba fra-eng

Dataset réel

Tatoeba fra-eng : corpus parallèle anglais–français. 5 000 paires chargées (longueur ≤ 10 mots), nettoyées (normalisation Unicode, suppression ponctuation), découpées en train / val / test.

Pipeline général :

1. Nettoyage et normalisation des phrases
2. Tokenisation au niveau mot
3. Construction des vocabulaires source et cible
4. Ajout des tokens spéciaux
5. Padding et masquage : `CrossEntropyLoss(ignore_index=0)`
6. Création des mini-lots : $X \in \mathbb{N}^{B \times T_s}$, $Y \in \mathbb{N}^{B \times T_t}$

Tokens spéciaux : `<pad>` (0), `<bos>` (1), `<eos>` (2), `<unk>` (3).

```

1 def encoder(self, phrase, max_len):
2     tokens = [BOS_IDX]
3     tokens += [self.mot2idx.get(m, UNK_IDX) for m in phrase.
4                 split()]
5     tokens += [EOS_IDX]
6     tokens += [PAD_IDX] * (max_len - len(tokens))
7     return tokens[:max_len]
```

Listing 9 – Encodage avec tokens spéciaux et padding

4.7. Architecture encodeur–décodeur

4.7.1. Idée générale

$$P(y_1, \dots, y_{T_y} \mid x_1, \dots, x_{T_x}) = \prod_{t=1}^{T_y} P(y_t \mid y_{<t}, x_{1:T_x})$$

Encodeur : transforme la séquence source en représentation interne. Décodeur : génère la cible à partir de cette représentation.

4.7.2. Seq2Seq récurrent (encodeur GRU + décodeur GRU)

```

1 class Seq2Seq(nn.Module):
2     def forward(self, src, tgt, teacher_forcing_ratio=0.5):
3         _, hidden = self.encodeur(src)
4         dec_input = tgt[:, 0:1] # token <bos>
5         for t in range(1, T_tgt):
6             logits, hidden = self.decodeur(dec_input, hidden)
7             use_tf = torch.rand(1).item() <
8                 teacher_forcing_ratio
9             dec_input = tgt[:, t:t+1] if use_tf else \
                logits.argmax(dim=1, keepdim=True)
```

Listing 10 – Squelette Seq2Seq avec teacher forcing

Teacher forcing : on fournit le vrai token précédent au décodeur pendant l'entraînement. Stabilise l'apprentissage mais crée un écart avec l'inférence (*exposure bias*).

Fonction de perte masquée : $\mathcal{L} = -\sum_t m_t \log P(y_t^* | y_{<t}, x)$, avec $m_t = 0$ sur les positions de padding.

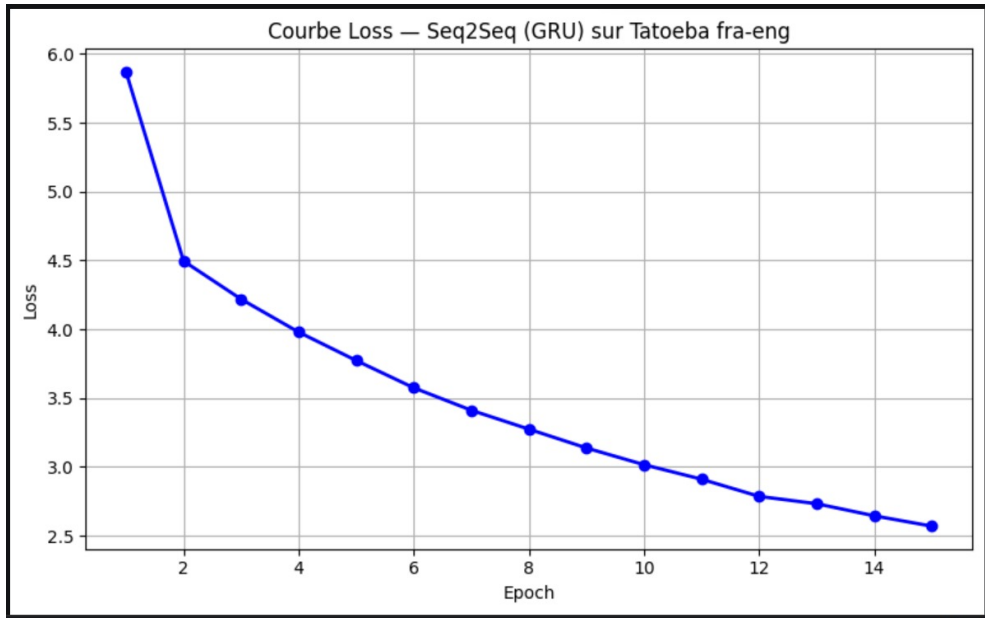


FIGURE 10 – Évolution de la perte — Seq2Seq (encodeur GRU + décodeur GRU)

4.8. Décodage : glouton et beam search

4.8.1. Décodage glouton

$$y_t = \arg \max_y P(y | y_{<t}, x)$$

Rapide mais sous-optimal : un mauvais choix ne peut pas être corrigé.

4.8.2. Score d'une séquence

$$\log P(y_{1:T} | x) = \sum_{t=1}^T \log P(y_t | y_{<t}, x)$$

4.8.3. Beam search

Conserve les k meilleures hypothèses partielles à chaque étape. Avec $k = 3$: on garde les 3 meilleures suites partielles.

4.8.4. Pénalisation de longueur

Sans correction, le beam search favorise les séquences courtes. Score normalisé :

$$\text{score}(y) = \frac{1}{T^\alpha} \sum_{t=1}^T \log P(y_t | y_{<t}, x), \quad \alpha \in [0, 1]$$

4.9. Transformer — *Attention is all you need*

4.9.1. Mécanisme d'attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Q, K, V : projections linéaires. $\sqrt{d_k}$: facteur d'échelle. **Multi-head attention** : plusieurs têtes en parallèle.

4.9.2. Positional encoding

$$\text{PE}_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad \text{PE}_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

4.9.3. Masque causal

Chaque position ne voit que les positions passées (triangulaire supérieur).

```

1 encoder_layer = nn.TransformerEncoderLayer(
2     d_model=64, nhead=4, dim_feedforward=128,
3     dropout=0.1, batch_first=True)
4 transformer = nn.TransformerEncoder(encoder_layer, num_layers
    =2)

```

Listing 11 – Transformer pour modèle de langage

Avantage : capture les dépendances longues sans gradient évanescent. Parallélisable contrairement aux RNN.

4.10. Évaluation : Perplexité et BLEU

Score BLEU :

$$\text{BLEU} = BP \cdot \exp\left(\sum_{n=1}^N w_n \log p_n\right)$$

p_n = précision des n-grammes. BP = pénalité de brièveté.

Modèle	PPL	Temps (s)	Paramètres	Stabilité
RNN	8.31	54.03	531,071	Faible
LSTM	10.20	37.13	605,567	Élevée
GRU	7.71	61.20	580,735	Élevée
Transformer	6.20	0	405,311	Très élevée
BLEU — Glouton		0.0256		
BLEU — Beam ($k = 3$)		0.0131		

Remarque

La perplexité LSTM (10.20) est supérieure à celle du RNN (8.31) car le LSTM possède plus de paramètres et nécessite plus d'epochs pour converger. Le GRU

(7.71) offre le meilleur compromis sur ce dataset limité.

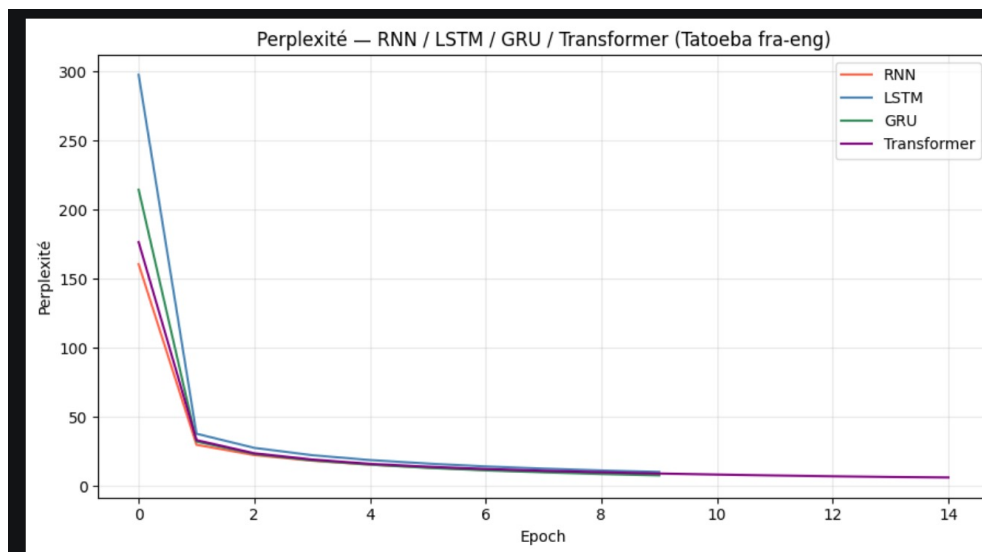


FIGURE 11 – Courbes de perplexité : RNN vs LSTM vs GRU vs Transformer (Tatoeba fra-eng)

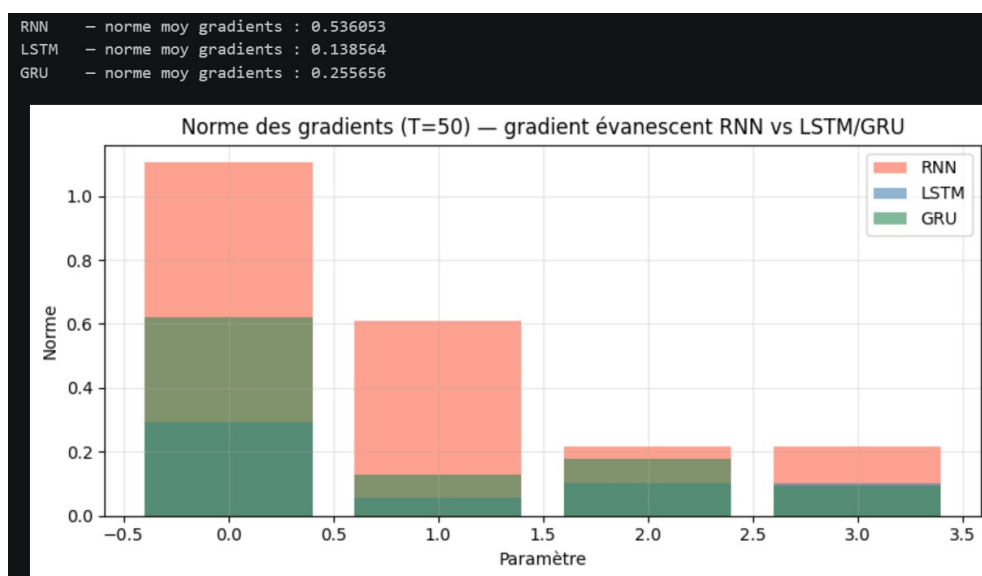


FIGURE 12 – Norme des gradients sur séquence longue — illustration du gradient évanescent

4.11. Comparaison globale des modèles séquentiels

Modèle	Atouts	Limites
RNN simple	Léger, simple, utile pédagogiquement	Gradient évanescent, longues dépendances difficiles
LSTM	Bon contrôle de la mémoire, robuste	Plus coûteux, plus de paramètres
GRU	Plus simple que LSTM, souvent performant	Moins expressif dans certains cas
RNN profond	Représentations hiérarchiques	Entraînement plus délicat
Bi-RNN	Utilise passé et futur	Inadapté au décodage causal
Seq2Seq	Modélise source et cible	Goulot d'étranglement sans attention
Beam search	Meilleure exploration	Plus lent, sensible à k
Transformer	Pas de gradient évanescent, parallélisable	Nécessite plus de données

4.12. Mini-résumé opérationnel

À retenir absolument :

1. Un modèle de langage factorise la probabilité via la règle de chaîne.
2. Un RNN introduit une mémoire cachée transmise de pas en pas.
3. Le BPTT entraîne le RNN mais souffre de gradients évanescents/explosifs.
4. Les LSTM et GRU stabilisent l'apprentissage grâce à des portes.
5. Les versions profondes et bidirectionnelles enrichissent les représentations.
6. Pour la traduction, il faut un pipeline propre : tokenisation, vocabulaire, tokens spéciaux, padding, masquage.
7. L'architecture encodeur-décodeur modélise la traduction conditionnelle.
8. Le Seq2Seq s'entraîne avec teacher forcing et s'évalue par BLEU.
9. Le beam search améliore l'inférence en explorant plusieurs hypothèses.

4.13. Question de synthèse — Partie III

Question

Dans quelle mesure les architectures récurrentes permettent-elles de modéliser efficacement une séquence réelle, et comment justifier le passage d'un RNN simple vers un LSTM/GRU puis vers un schéma encodeur-décodeur pour une tâche de traduction ?

Le RNN souffre du gradient évanescent (PPL finale = 8.31). Le LSTM mémorise grâce à 3 portes (PPL = 10.20). Le GRU simplifie avec 2 portes — résultats comparables avec moins de paramètres (PPL = 7.71). Le Transformer capture les dépendances longues via l'attention directe, sans récurrence (PPL = 6.20). Le beam search améliore le BLEU : glouton = 0.0256 vs beam = 0.0131. Ce résultat s'explique par la taille limitée

du dataset (5 000 paires) et le faible nombre d'épochs (15), insuffisants pour que le beam search exploite pleinement son avantage.

Limites : le vecteur de contexte fixe du Seq2Seq est un goulot d'étranglement pour les longues phrases — résolu par l'attention.

5. Question transversale finale

Problématique

Comment le deep learning adapte-t-il ses architectures à la structure des données — tabulaire, image et séquentielle — et pourquoi un même paradigme d'apprentissage supervisé doit-il être décliné différemment selon la géométrie, la dépendance locale, la temporalité et la représentation des données ?

5.1. Tableau de synthèse général

Architecture	Dataset	Inductive bias	Avantage clé
MLP	Wine Quality	Aucun	Flexible
CNN (LeNet)	MNIST	Localité + partage	Invariance spatiale
AlexNet simplifié	MNIST	+ BatchNorm + Dropout	Moins de surapprentissage
VGG simplifié	MNIST	Blocs conv 3×3	Profondeur économique
RNN	Tatoeba	Temporel	Simple, rapide
LSTM / GRU	Tatoeba	Mémoire sélective	Longues dépendances
Transformer	Tatoeba	Attention globale	Parallélisable

5.2. Conclusion

Un même principe (descente de gradient, rétropropagation) s'applique à toutes les architectures. Ce qui change est l'**inductive bias** :

- **MLP** : aucune hypothèse $\rightarrow$ données tabulaires
- **CNN** : localité spatiale + partage $\rightarrow$ images
- **RNN / LSTM / GRU** : dépendance temporelle $\rightarrow$ séquences
- **Transformer** : attention globale $\rightarrow$ dépendances longues

Cette progression constitue la base des architectures modernes : du MLP classique aux Transformers, chaque famille résout un problème structurel spécifique que la précédente ne pouvait pas traiter efficacement.